SYSTEM ANALYSIS DOCUMENTATION (SAD)

UMHackathon 2026

umhackathon@um.edu.my

© UMHackathon 2026

Email: umhackathon@um.edu.my

1. Introduction:

GridGuard is an agentic AI system designed for autonomous municipal hazard resolution. It addresses the inefficiencies of manual infrastructure maintenance such as roads, streetlights, and drainage where delayed resolutions endanger public safety. This section introduces the strategic solution that replaces fragmented, manual touchpoints with an automated, AI-driven response system.

1.1. Purpose:

The system analysis document highlights the technical scope and design-related decisions behind the GridGuard development. Key elements covered include:

So, the key element of system that has been covered are as followed:

1. **Architecture**:
    a. A stateful agentic workflow engine orchestrated by LangGraph. The services are containerized using Docker Compose and split between a React frontend and a FastAPI backend.
2. **Data Flows:**
    a. Hazard reports are ingested, classified, checked against a DuckDB operational budget, and recorded in a PostgreSQL audit trail.
3. **Model Process:**
    a. A cyclic reasoning process driven by GLM-5.1 that handles Triage, Financial Assessment, and Contractor Dispatch.
4. **Role of Reference:**
    a. This document serves as a reference for developers to understand the architecture, API contracts, and deterministic fallback mechanics

1.2. Background:

Municipalities currently face "silent failures" where reports are lost in fragmented manual chains involving classification, finance verification, and contractor coordination. This creates latency, especially when budget shortfalls or contractor unavailability occur.

© UMHackathon 2026

Email: umhackathon@um.edu.my

1.3. Target Stakeholder

Stakeholders	Roles	Expectations
Citizen	Reports hazards via photo and description.	Rapid, automated resolution without manual follow-up.
City Operations Manager	Reviews triaged incident summaries.	Single-action approval or rejection of AI-triaged reports.
Contractors	Receives dispatch orders	Clear task details and optimized navigation.

2. System Architecture & Design

2.1. High Level Architecture

2.1.1. Overview:

Type	Details
System	Web Dashboard (React + Vite + Tailwind)
Architecture	FastAPI Backend with LangGraph Orchestration

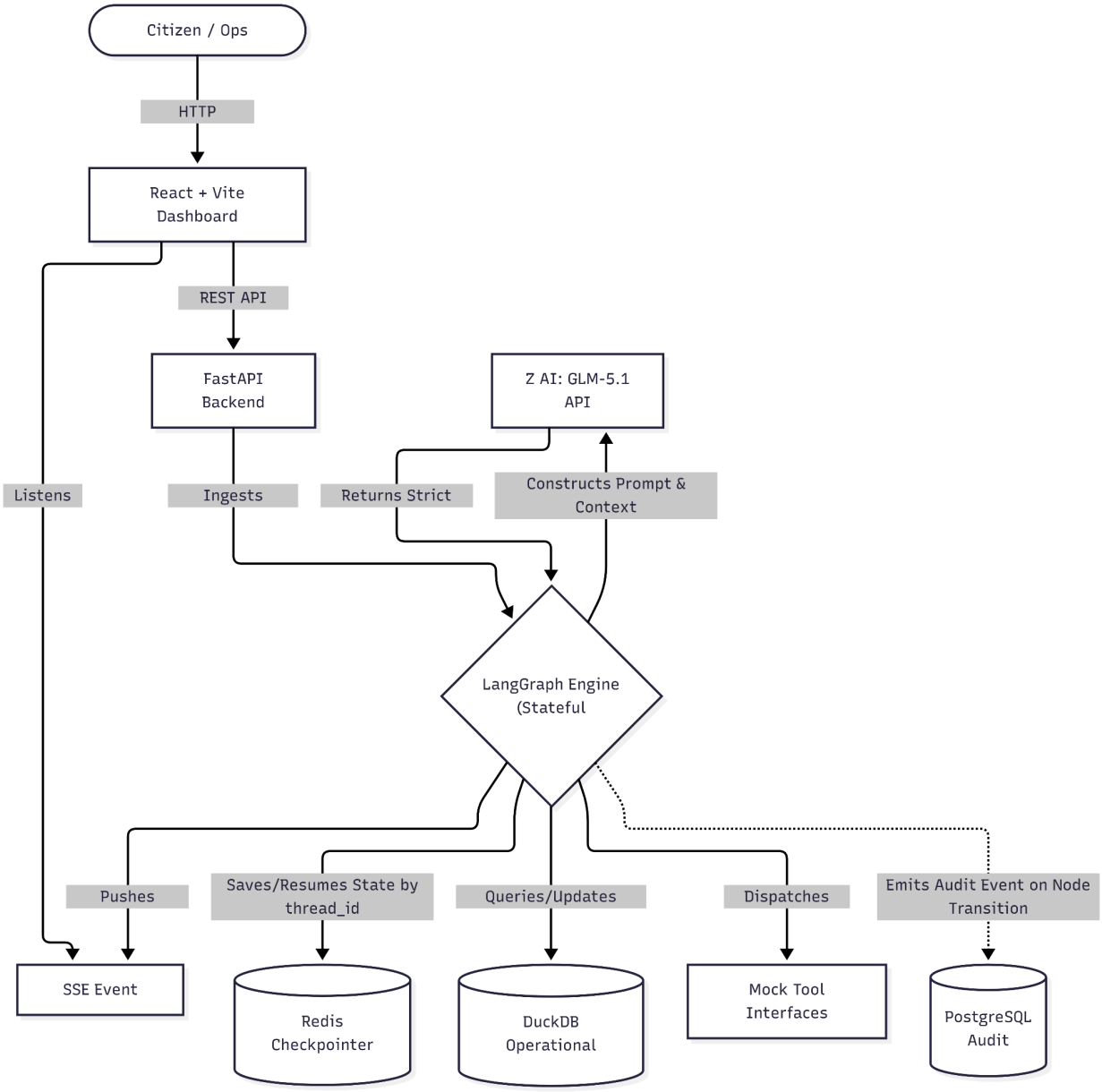
2.1.2 LLM as Service Layer

GridGuard integrates GLM-5.1 as a core reasoning engine rather than a generic module.

- **Prompt Construction**: The system constructs prompts by injecting the incident status, current budget levels (from DuckDB), and contractor availability.
- **Context Window:** Includes incident history and previous tool call outputs to maintain state.
- **Parsing**: The system enforces strict JSON outputs (e.g., TRANSFER_FUNDS, CONTINUE, ESCALATE) to drive the next node in the graph.
- **Token Management**: Inputs are truncated based on incident priority to ensure low latency.

© UMHackathon 2026

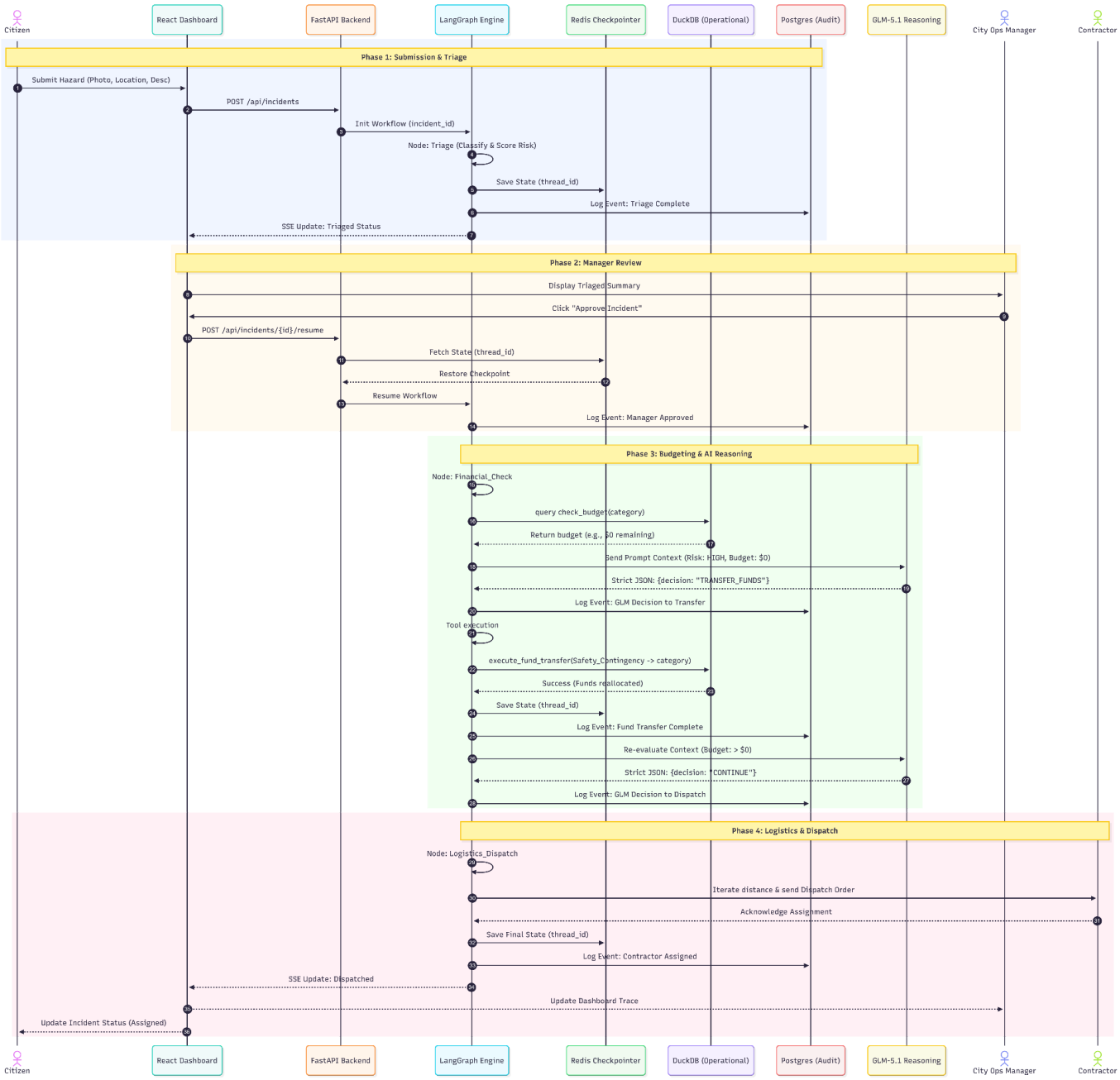
2.1.3 Dependency Diagram



© UMHackathon 2026

Email: umhackathon@um.edu.my

2.2.2. Sequence Diagram



© UMHackathon 2026

Email: umhackathon@um.edu.my

2.2. Technological Stack

2.2.1. Frontend: Nextjs + Vite + Tailwind

2.2.2. Backend: FastAPI (Python 3.12) and LangGraph

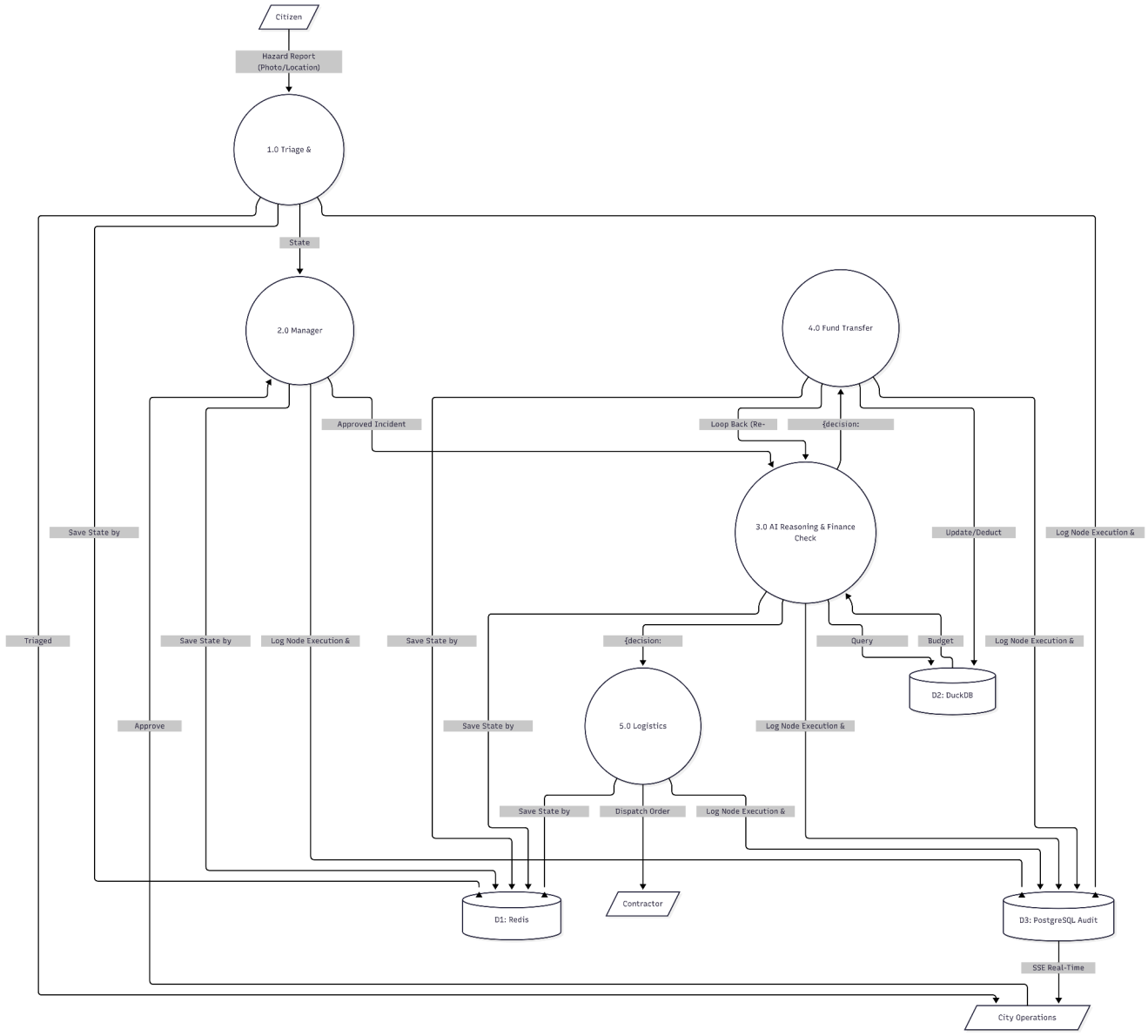
2.2.3. Database: **DuckDB** for operational budgeting and analytics, PostgreSQL for Audit logs and incident persistence and Redis for state checkpointer for durable execution.

2.2.4. Cloud/Deployment: Docker Compose (containerized stack).

2.3. Key Data Flows

Showing the how data are moving through the system and how its structure and stored

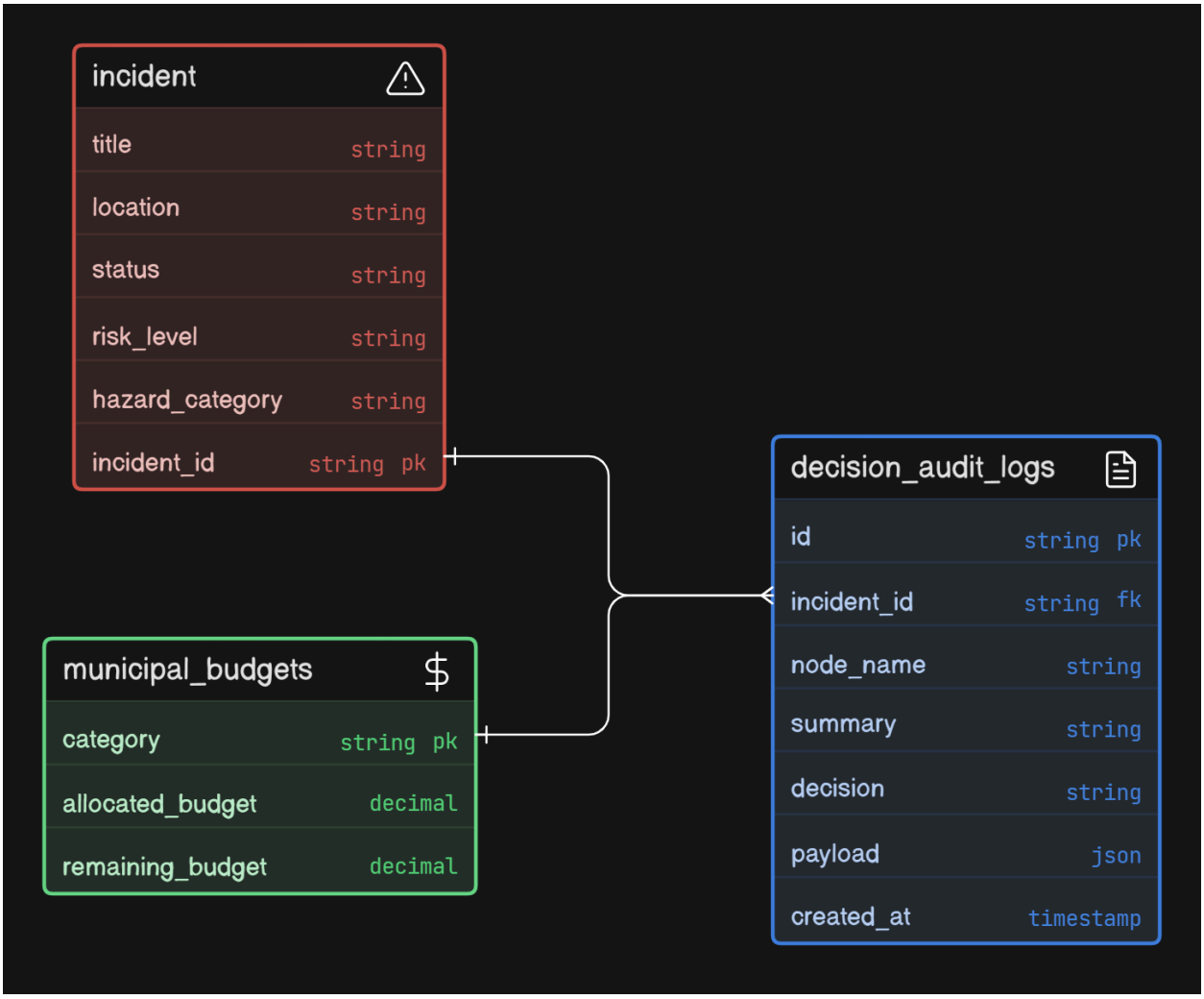
2.3.1.1. Data Flow Diagram (DFD)



© UMHackathon 2026

Email: umhackathon@um.edu.my

2.3.2. Normalized Database Schema



© UMHackathon 2026

Email: umhackathon@um.edu.my

3. Functional Requirements & Scope

This section highlights the core boundaries of the projects. That enables them to focus on a high-impact MVP to showcase the technical feasibility of the project.

3.1. Minimum Viable Product:

3.1.1. Do list down 3-5 core features that the team is going to build and do a demo by the end of the UMHackathon 2026. **[e.g. in Table]**

#	Features	Description
1	Citizen submit a hazard incident	Citizens may file a complaint by describing the hazard incident to the GridGuard.
2	Perform AI-triage incident & risk classification	After citizens file a complaint, GridGuard will classify the incident by hazard categories and allocate the budget based on the incident.
3	Contractors assigned	Based on the location and availability of contractors, GridGuard will assign a contractor for the incident to resolve the issue.
4	City Operation Managers Decide to Approve or Not	After GridGuard make a summary on the incidents, managers can choose to approve or not based on the summary given that include budget, hazard category and location.

3.2. Non-Functional Requirements (NFRs)

Here, highlights the system qualities that are not features but are crucial for reliability and improve the system.

Quality (e.g.)	Requirements (e.g.)	Implementation (e.g)
Scalability	Handle concurrent incident reports.	Async FastAPI routes and modular micro-services.
Reliability	Durable execution even if the server restarts.	Redis checkpointer saves graph state by `thread_id`.
Auditability	Every AI decision must be explainable.	Deterministic JSON outputs stored in PostgreSQL.

© UMHackathon 2026

Latency	GLM reasoning should complete under 5s.	Fixed-length prompt construction and optimized context.
Maintainability	The backend should be modular and easy to extend.	Separate api, db, graph, models, services, and tools layers.

3.3. Out of Scope / Future Enhancements

a. During submission of complaint, the In-App chat features between the citizen and customer service in city operation manager office
b. Estimated time and current status of the incident will be resolved.
c. Real time contractor location near the incident.

4. Monitor, Evaluation, Assumptions & Dependencies

4.1. Technical Evaluation:

4.1.1. **Grayscale Rollout & A/B Testing**:Gridguard will be tested to only selected municipal governments and then it will be gradually increased from one to the other district. If no issues are detected until 80% of districts selected in a state, it will be released to all municipal governments. For the citizen to submit the hazard incident, it will be released fully from the start, but the applications can only function properly for selected districts only.

4.1.2. **Emergency Rollback (ER) & Golden Release**: If the GLM API fails, the system transitions to a HUMAN_ESCALATION state rather than silent failure. This is to prevent the issue being unresolvable and give back the responsibility to humans.

4.1.3. Priority Matrix:

4.1.3.1. Hazards with "CRITICAL" risk levels trigger immediate budget reallocation regardless of category limits. The higher the levels of hazard it will be more prioritized and more budget will be allocated.

4.2. Monitoring:

4.2.1. GridGuard should monitor workflow success rate, budget transfer frequency, contractor retry count, escalation rate, and response latency. These metrics allow the team to detect whether the autonomous loop is resolving incidents efficiently or relying too often on escalation.

© UMHackathon 2026

4.2.2. A practical evaluation approach is to test high-risk, zero-budget incidents, unavailable-contractor scenarios, and interrupted executions. These cases validate the key MVP promises: cyclic reasoning, budget reallocation, retry fallback, and durable resume.

4.3. **Assumptions:** Key operational & environmental conditions that are assumed to be valid while development of system and deployment.

- **a.** Citizen has a good camera quality to capture picture of the incidents
- **b.** Municipal government have a good computer to run the website
- **c.** Contractor always activate their current locations
- **d.** The incidents that happen are witnessed by citizens who's aware of these applications.

4.4. **External Dependencies:** All the external tools that are applied to the system to function.

Tools	Purpose	Risks
Z Ai GLM API	The core LLM reasoning engine - that will handle the prompt inference, multi-step reasoning and generating structured output	High- in case of GLM API rate limitation - core product will fail. So, implement a mitigation strategy [e.g. graceful degradation state from users such as manual escalation by humans.
Google Maps API	Tracking contractors location and the incident location distance	Medium level- if real time tracking is being disabled.

5. Project Management & Team Contributions

The duration of the Preliminary round is approximately 10 days considering the two-week sprint.

5.1. Project Timeline:

Day 1 - 3: Discussion on what project that we want to develop. Identify the problem statement and example of the situation.

Day 4 - 5: Make some research on what tools to be used, framework and features in the website. Find any tutorials and documentation to help complete the project.

Day 6 - 8: The development of the project started on both frontend and backend to test the Z.AI model capabilities.

© UMHackathon 2026

Day 9 - 10: Started to focus on documentation, testing and finalised the prototype.

5.2. Team Members Role:

Umar Syakir - Hackathon project leader and secondary backend dev
Hafizul Rahman - Lead backend dev
Shahril Haikal - Lead frontend dev
Akmal Fikri - Lead project documentation and pitching

5.3. Recommendations: GridGuard should keep the reasoning schema versioned so future workflow changes remain compatible with old traces. It should also separate budget-reconciliation logic from dispatch logic to make testing easier and audit events more consistent.

For operational maturity, the team should add rate-limited retries, structured error codes, and dashboard alerts for repeated escalation patterns. These improvements would strengthen reliability without changing the core MVP scope

© UMHackathon 2026